

Programadores como Guardianes de la Salud del Código

Artie Shevchenko

El nuevo rol: Code Health Guardian

Artie Shevchenko, ingeniero en Canva y profesor en la Universidad ITMO, propone que los programadores adopten un rol clave en la era de la IA: el Code Health Guardian. Según él, tres habilidades son esenciales para tener éxito: colaborar con la IA, delegar en la IA y convertirse en expertos en la gestión de la complejidad del código, ya que el razonamiento complejo sigue siendo una ventaja humana frente a la IA.

La salud del código

La salud del código no es solo simplicidad o mantenibilidad: es, en el fondo, una cuestión de productividad. El verdadero reto está en encontrar el equilibrio entre productividad a corto y largo plazo: entregar valor ahora sin comprometer la capacidad de evolucionar el sistema en el futuro.

Los 3 problemas de la complejidad

- **Change Amplification:** un cambio pequeño acaba afectando cientos de líneas y múltiples archivos, ralentizando el desarrollo.
- **Cognitive Load:** esfuerzo mental excesivo para entender el código antes de poder modificarlo, lo que genera agotamiento y errores.
- **Unknown Unknowns:** el problema más grave; el desarrollador no sabe lo que no sabe, lo que puede provocar fallos graves e imprevisibles.

Causas principales de la complejidad

- **Obscuridad:** nombres poco claros, falta de comentarios y estructuras difíciles de rastrear. Es la causa más importante.
- **Interfaces complejas:** APIs con demasiados parámetros o excesivamente genéricas que generan confusión y aumentan la carga cognitiva.
- **Falta de consistencia (Unfamiliarity):** cada desarrollador con estilos distintos dificulta el mantenimiento del sistema.

Principios y Procesos para un Código Sano

Design by Contract

Un módulo (interfaz + implementación) debe tener contratos claros: precondiciones, postcondiciones e invariantes de clase. No basta con los tests, ya que en producción existen millones de estados posibles. Se recomienda el uso de assertions en runtime para validar precondiciones y fallar rápido. Un principio cultural clave: no tragarse excepciones, sino fallar explícitamente.

DRY desde la perspectiva de la complejidad

La duplicación de código abarca los tres tipos de complejidad. Artie recomienda no obsesionarse con eliminar cada duplicado, sino aplicar el DRY de forma pragmática:

- Primero, eliminar los unknown unknowns.
- Luego, evaluar el coste-beneficio de abstraer el código duplicado.
- Por último, eliminar duplicación solo si aporta simplicidad real.

Design Docs

No hace falta un gran diseño inicial. Un diseño mínimo pero consciente antes de implementar es suficiente y compatible con metodologías ágiles. Estos documentos ligeros contienen contexto, objetivos, una vista general de la solución y, sobre todo, un análisis de trade-offs (pros y contras de las distintas opciones). Su objetivo es solicitar feedback y reducir el retrabajo.

Revisión de Código

Las revisiones de código no deben centrarse solo en la calidad técnica. Artie propone cuatro objetivos:

- **Calidad de código:** detección de fallos y mejora del diseño.
- **Compartir conocimiento:** el revisor ve el contexto real del código y se produce aprendizaje concreto.
- **Velocidad de entrega:** optimizar las revisiones para no caer en el perfeccionismo.
- **Seguridad psicológica:** un clima seguro permite reconocer errores, facilitar el aprendizaje y aumentar la productividad del equipo.

Conclusión

La complejidad en el software no es un problema único, sino un conjunto de factores interrelacionados que afectan tanto al producto como al equipo. Más complejidad genera más errores, los errores más parches, y los parches más complejidad. Comprender estos ciclos y aplicar principios como Design by Contract, DRY pragmático, Design Docs y revisiones de código holísticas es fundamental para mantener un sistema sostenible a largo plazo.